Django Database & ORM — Complete Notes A to Z

Version: Django 6.0

Level: Beginner to Advanced

Language: Simple English + Bangla Explanation

Table of Contents

1. Database Basics
2. Django ORM
3. Project Setup
4. Django Models and Fields
5. Migrations
6. CRUD Operations
7. QuerySets and Filtering
8. Field Lookups and Q Objects
9. Model Relationships
10. Ordering and Limiting Results
11. `select_related()` and `prefetch_related()`
12. Aggregation and Annotation
13. F Expressions
14. Conditional Expressions
15. Subquery and Exists
16. Managers and Custom QuerySets
17. Database Transactions
18. Raw SQL
19. Query Optimization and Indexes
20. Bulk Operations
21. Multiple Databases
22. Fixtures
23. Django Signals
24. Abstract Models and Proxy Models
25. Validation, Constraints and Security
26. Python and Django Introspection

27. Interview Questions
28. Quick ORM Cheat Sheet
29. Learning Roadmap
30. Capstone Project

Chapter 1: Database Basics

1.1 What Is a Database?

A database is a system for storing, finding, updating, and deleting information.

Examples:

- Student names and emails
- Product information
- Customer orders
- User accounts
- Examination results

বাংলা: Database হলো এমন একটি ব্যবস্থা যেখানে তথ্য সংরক্ষণ, খোঁজা, পরিবর্তন এবং মুছে ফেলা যায়।

1.2 What Is a Relational Database?

A relational database stores information in tables containing rows and columns.

Popular relational databases:

- PostgreSQL
- MySQL
- SQLite
- Oracle

Term	Meaning
Table	A collection of related data
Row	One record

Column	One field or attribute
Primary key	Uniquely identifies a record
Foreign key	Connects records across tables
Constraint	A rule that data must follow
Index	Can help the database find data faster

Example:

id	name	email
1	Rahim	rahim@example.com
2	Karim	karim@example.com

Here, `id` can be the primary key.

বাংলা: Primary key প্রতিটি record-কে আলাদাভাবে শনাক্ত করে। Foreign key একটি table-এর record-কে অন্য table-এর record-এর সঙ্গে যুক্ত করে।

1.3 What Is SQL?

SQL means Structured Query Language. It is used to work with relational databases.

```
SELECT * FROM students_student;

SELECT *
FROM students_student
WHERE age >= 18;

INSERT INTO students_student (name, age)
VALUES ('Rahim', 20);

UPDATE students_student
SET age = 21
WHERE id = 1;

DELETE FROM students_student
WHERE id = 1;
```

The table name depends on the model and database configuration.

1.4 What Is ACID?

ACID describes important properties of database transactions.

- **Atomicity:** All operations succeed, or the transaction is rolled back.
- **Consistency:** Data follows database rules.
- **Isolation:** Concurrent transactions follow the database's isolation rules.
- **Durability:** Committed changes are preserved according to the database's guarantees.

Example: A bank transfer should not subtract money from one account without adding it to the other.

Chapter 2: Django ORM

2.1 What Is ORM?

ORM means Object-Relational Mapping.

Django ORM allows you to work with a relational database using Python models and query methods instead of writing SQL manually for every operation.

SQL:

```
SELECT *
FROM students_student
WHERE age >= 18;
```

Django ORM:

```
Student.objects.filter(age__gte=18)
```

বাংলা: ORM Python object ও database table-এর মধ্যে সংযোগ তৈরি করে। প্রতিটি query হাতে SQL লিখতে হয় না।

2.2 Benefits of Django ORM

- Write database queries using Python.
- Define tables through models.
- Manage schema changes through migrations.
- Work with related models.
- Build queries by chaining QuerySets.
- Use parameterized queries for many database operations.
- Switch database backends more easily, although features can differ between backends.

2.3 View the SQL Query

```
students = Student.objects.filter(age__gte=18)

print(students.query)
```

This displays the SQL representation of the QuerySet. It does not normally execute the query by itself.

বাংলা: ORM কী SQL তৈরি করছে তা দেখলে query debugging ও performance সমস্যা খুঁজে বের করা সহজ হয়।

Chapter 3: Project Setup

3.1 Create a Virtual Environment

```
python -m venv .venv
```

Windows:

```
.venv\Scripts\activate
```

macOS/Linux:

```
source .venv/bin/activate
```

3.2 Install Django

For a Django 6.0 project:

```
python -m pip install "Django>=6.0,<6.1"
```

Check the installed version:

```
python -m django --version
```

3.3 Create a Project and App

```
django-admin startproject config .
python manage.py startapp students
```

Register the app in `config/settings.py`:

```
INSTALLED_APPS = [
    # Keep the default Django applications.
    "students",
]
```

Add `"students"` to the existing list; do not remove the default Django applications.

3.4 Apply Initial Migrations and Run the Server

```
python manage.py migrate
python manage.py runserver
```

3.5 SQLite Configuration

A new Django project normally uses SQLite.

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

SQLite is useful for learning and many small projects.

3.6 PostgreSQL Configuration

Install a PostgreSQL driver compatible with your environment.

Keep credentials in environment variables or a secret manager.

```
import os

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ["DB_NAME"],
        "USER": os.environ["DB_USER"],
        "PASSWORD": os.environ["DB_PASSWORD"],
        "HOST": os.environ.get("DB_HOST", "localhost"),
        "PORT": os.environ.get("DB_PORT", "5432"),
    }
}
```

বাংলা: Local learning-এর জন্য SQLite সহজ। Production application-এ PostgreSQL বহুল ব্যবহৃত।

Database password Git-এ commit করবে না।

Chapter 4: Django Models and Fields

4.1 What Is a Model?

A model is a Python class that describes data stored in a database table.

In `students/models.py`:

```
from django.db import models


class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    age = models.PositiveIntegerField(default=18)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

Explanation:

- `models.Model` : makes the class a Django model.
- `CharField` : stores limited-length text.

- `EmailField` : provides email-oriented validation.
- `unique=True` : prevents duplicate email values at the database level.
- `default=18` : supplies a default value.
- `auto_now_add=True` : sets the timestamp when the object is first created.
- `__str__()` : returns a human-readable representation.

Django normally adds a primary key automatically if one is not defined.

বাংলা: Model হলো database table-এর Python representation। Field দিয়ে table-এর column ও তার type নির্ধারণ করা হয়।

4.2 Common Model Fields

Field	Use
`CharField(max_length=...)`	Limited-length text
`TextField()`	Longer text
`IntegerField()`	Integer values
`PositiveIntegerField()`	Nonnegative integer-oriented values
`DecimalField(max_digits=10, decimal_places=2)`	Precise decimal values, such as money
`FloatField()`	Floating-point

	values
`BooleanField()`	True/False
`DateField()`	Date
`DateTimeField()`	Date and time
`EmailField()`	Email
`URLField()`	URL
`UUIDField()`	UUID
`JSONField()`	JSON data
`FileField(upload_to="files/")`	File path/reference
`ImageField(upload_to="images/")`	Image path/reference and image validation support

Use `DecimalField` for money when decimal precision matters. Floating-point values can introduce precision differences.

4.3 `null=True` vs `blank=True`

```
middle_name = models.CharField(
    max_length=100,
```

```
    blank=True,
)

age = models.IntegerField(
    null=True,
    blank=True,
)
```

- `null=True` : the database can store SQL `NULL` .
- `blank=True` : the field may be empty during Django validation.

বাংলা: `null` database storage-এর বিষয়; `blank` validation-এর বিষয়। দুটো এক নয়।

For string fields, Django projects usually prefer an empty string (`""`) rather than using both `NULL` and an empty string to represent missing text.

4.4 `default` , `unique` , and `db_index`

```
status = models.CharField(
    max_length=20,
    default="pending",
)

email = models.EmailField(unique=True)

slug = models.SlugField(db_index=True)
```

- `default` : value না দিলে কোন মান ব্যবহার হবে।
- `unique=True` : duplicate value আটকায়।
- `db_index=True` : field-এর ওপর database index তৈরি করে।

Indexes can improve some reads, but they use storage and may slow down writes.

4.5 Model Meta Options

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["name"]
        verbose_name = "Student"
        verbose_name_plural = "Students"
        indexes = [
            models.Index(fields=["created_at"]),
        ]
```

- `ordering` : default ordering of results.
- `verbose_name` : singular display name.
- `verbose_name_plural` : plural display name.
- `indexes` : database indexes declared for the model.

4.6 Model Methods

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    def __str__(self):
        return self.name

    def is_expensive(self):
        return self.price >= 1000
```

`is_expensive()` is a Python method, not a database field.

4.7 Choices

```
class Order(models.Model):
    class Status(models.TextChoices):
        PENDING = "pending", "Pending"
        PAID = "paid", "Paid"
        CANCELLED = "cancelled", "Cancelled"

    status = models.CharField(
        max_length=20,
        choices=Status.choices,
        default=Status.PENDING,
    )
```

The first value is stored in the database; the second is the display label.
Choices help validation and display. Use database constraints when you need a database-level rule.

Chapter 5: Migrations

5.1 What Is a Migration?

A migration is a version-controlled instruction for changing database structure.

If you add a model field, a migration can add the corresponding database column.

5.2 Important Commands

```
python manage.py makemigrations
python manage.py migrate
python manage.py showmigrations
python manage.py sqlmigrate students 0001
```

Command	Purpose
`makemigrations`	Creates migration files from model changes
`migrate`	Applies migrations to the database
`showmigrations`	Shows migration status
`sqlmigrate`	Shows SQL for a migration

বাংলা: `makemigrations` file তৈরি করে; `migrate` database-এ পরিবর্তন প্রয়োগ করে। দুটি command একই কাজ করে না।

5.3 Migration Workflow

1. Change `models.py`.

2. Run `makemigrations`.
3. Review the generated migration.
4. Run `migrate`.
5. Run tests and verify the result.

Keep migration files in version control. Before production schema changes, consider backups, data migrations, deployment order, and rollback strategy.

Chapter 6: CRUD Operations

CRUD means Create, Read, Update, and Delete.

6.1 Create

```
student = Student.objects.create(
    name="Rahim",
    email="rahim@example.com",
    age=20,
)
```

Alternative:

```
student = Student(
    name="Karim",
    email="karim@example.com",
    age=21,
)

student.save()
```

`create()` creates and saves an object. Instantiating a model does not save it until `save()` is called.

6.2 Read

```
Student.objects.all()

Student.objects.get(pk=1)

Student.objects.filter(is_active=True)

Student.objects.exclude(age__lt=18)
```

- `all()`: returns all records in the QuerySet.

- `get()` : expects exactly one object.
- `filter()` : returns a QuerySet of matches.
- `exclude()` : returns a QuerySet excluding matches.

`get()` raises `DoesNotExist` if no object matches and `MultipleObjectsReturned` if multiple objects match.

```
try:
    student = Student.objects.get(pk=1)
except Student.DoesNotExist:
    student = None
```

6.3 Update One Object

```
student = Student.objects.get(pk=1)

student.age = 22

student.save(update_fields=["age"])
```

6.4 Update Multiple Objects

```
Student.objects.filter(
    is_active=False
).update(is_active=True)
```

`QuerySet.update()` changes matching rows directly. It does not call each instance's `save()` method or normal save signals.

6.5 Delete

```
student = Student.objects.get(pk=1)
student.delete()
```

Delete multiple objects:

```
Student.objects.filter(is_active=False).delete()
```

Always check the filter before deleting data. Django's deletion collector may also delete related objects according to relationship rules.

6.6 `get_or_create()`

```
student, created = Student.objects.get_or_create(
    email="rahim@example.com",
```

```
    defaults={
        "name": "Rahim",
        "age": 20,
    },
)
```

It returns a matching object or creates one. `created` indicates whether a new object was created.

Database uniqueness constraints remain important under concurrent requests.

6.7 `update_or_create()`

```
student, created = Student.objects.update_or_create(
    email="rahim@example.com",
    defaults={
        "name": "Rahim Updated",
        "age": 21,
    },
)
```

It updates a matching object or creates one if none matches.

বাংলা: CRUD হলো database কাজের ভিত্তি। `get()` একটি object-এর জন্য, `filter()` এক বা একাধিক result-এর জন্য ব্যবহার করো।

Chapter 7: QuerySets and Filtering

7.1 What Is a QuerySet?

A QuerySet represents a database query and its results.

```
students = Student.objects.filter(age__gte=18)
```

QuerySets are usually lazy: creating one does not normally execute the database query immediately.

7.2 QuerySet Chaining

```
students = (
    Student.objects
    .filter(is_active=True)
    .filter(age__gte=18)
    .order_by("name")
)
```

Multiple filter conditions are normally combined using AND.

7.3 Common Filters

```
Student.objects.filter(name="Rahim")

Student.objects.filter(age__gte=18)

Student.objects.filter(age__range=(18, 25))

Student.objects.filter(name__startswith="R")

Student.objects.filter(
    email__icontains="example.com"
)

Student.objects.filter(
    created_at__isnull=False
)
```

7.4 QuerySet Evaluation

Operations that can evaluate a QuerySet include:

```
list(students)

len(students)

bool(students)

for student in students:
    print(student.name)
```

An evaluated QuerySet may cache its results. A newly constructed QuerySet does not automatically share that cache.

Use `exists()` when you only need to know whether matches exist, and `count()` when you need the number of matching rows.

বাংলা: QuerySet lazy হওয়ায় query প্রয়োজন হওয়ার আগে database access নাও হতে পারে। এতে query chain করে একসঙ্গে শর্ত তৈরি করা যায়।

Chapter 8: Field Lookups and Q Objects

8.1 Common Field Lookups

General pattern:

```
Model.objects.filter(field__lookup=value)
```

Lookup	Meaning	Example
`exact`	Exact match	`name__exact="Rahim"`
`iexact`	Case-insensitive exact match	`name__iexact="rahim"`
`contains`	Contains text	`name__contains="ah"`
`icontains`	Case-insensitive contains	`name__icontains="rah"`
`gt`	Greater than	`age__gt=18`
`gte`	Greater than or equal	`age__gte=18`
`lt`	Less than	`age__lt=18`
`lte`	Less than or equal	`age__lte=18`
`in`	Value is in a list	`id__in=[1, 2, 3]`
`range`	Value is within a	`age__range=(18,`

	range	`25)`
`isnull`	Checks SQL NULL	`created_at__isnull=True`
`startswith`	Starts with text	`name__startswith="R"`
`endswith`	Ends with text	`name__endswith="m"`

8.2 Q Objects

Q objects help build complex conditions.

```
from django.db.models import Q
```

OR:

```
students = Student.objects.filter(
    Q(age__lt=18) | Q(age__gt=60)
)
```

AND:

```
students = Student.objects.filter(
    Q(is_active=True) & Q(age__gte=18)
)
```

NOT:

```
students = Student.objects.filter(
    ~Q(name__icontains="test")
)
```

- `|` means OR.
- `&` means AND.
- `~` means NOT.

Example: Search by name or email.

```
search = "rahim"

students = Student.objects.filter(
    Q(name__icontains=search)
    | Q(email__icontains=search)
)
```

বাংলা: সাধারণ filter arguments সাধারণত AND হিসেবে যুক্ত হয়। OR/NOT condition-এর জন্য Q object খুব কাজে লাগে।

Chapter 9: Model Relationships

9.1 ForeignKey — One-to-Many

One author can have many posts, while each post refers to one author.

```
class Author(models.Model):
    name = models.CharField(max_length=100)

    def __str__(self):
        return self.name


class Post(models.Model):
    author = models.ForeignKey(
        Author,
        on_delete=models.CASCADE,
        related_name="posts",
    )
    title = models.CharField(max_length=200)
    body = models.TextField()
```

Usage:

```
author = Author.objects.get(pk=1)

author.posts.all()

post = Post.objects.get(pk=1)

post.author
```

`on_delete` Options

- `CASCADE` : deletes related objects when the referenced object is deleted.

- `PROTECT`: prevents deletion when protected related objects exist.
- `RESTRICT`: restricts deletion under relationship conditions.
- `SET_NULL`: sets the foreign key to `NULL`; requires `null=True`.
- `SET_DEFAULT`: sets the foreign key to its default.
- `DO_NOTHING`: Django takes no automatic action; the database may raise an error.

Choose the behavior based on business requirements.

9.2 OneToOneField

```
class Profile(models.Model):
    student = models.OneToOneField(
        Student,
        on_delete=models.CASCADE,
        related_name="profile",
    )
    bio = models.TextField(blank=True)
```

Usage:

```
student = Student.objects.get(pk=1)

profile = student.profile
```

Each `Student` can have at most one `Profile` through this relationship. Reverse access can raise a related-object exception if no profile exists.

9.3 ManyToManyField

A student can take many courses, and a course can have many students.

```
class Course(models.Model):
    title = models.CharField(max_length=100)


class Learner(models.Model):
    name = models.CharField(max_length=100)
    courses = models.ManyToManyField(
        Course,
        related_name="learners",
        blank=True,
    )
```

Usage:

```
learner = Learner.objects.get(pk=1)
course = Course.objects.get(pk=1)

learner.courses.add(course)

learner.courses.remove(course)

learner.courses.set([course])

learner.courses.clear()
```

Django normally creates an intermediary table for a Many-to-Many relationship.

9.4 Through Model

Use a through model when the relationship has its own data, such as enrollment date or grade.

```
class Enrollment(models.Model):
    learner = models.ForeignKey(
        Learner,
        on_delete=models.CASCADE,
    )
    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE,
    )
    enrolled_at = models.DateTimeField(auto_now_add=True)
    grade = models.CharField(max_length=5, blank=True)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["learner", "course"],
                name="unique_learner_course",
            )
        ]
```

Connect it to a Many-to-Many field using `through="Enrollment"` when defining the relationship.

বাংলা:

- `ForeignKey` : সাধারণত one-to-many।
- `OneToOneField` : one-to-one।

- `ManyToManyField` : many-to-many

Chapter 10: Ordering and Limiting Results

10.1 Ordering

```
Student.objects.order_by("name")

Student.objects.order_by("-created_at")

Student.objects.order_by("name", "-age")
```

A minus sign means descending order.

10.2 Limiting Results

```
Student.objects.all()[:10]

Student.objects.all()[10:20]
```

These slices normally translate to SQL `LIMIT` and `OFFSET` .

Large offsets can become expensive; cursor or keyset pagination may be useful for some workloads.

10.3 `values()` and `values_list()`

```
Student.objects.values("id", "name", "email")

Student.objects.values_list("name", flat=True)
```

- `values()` returns dictionaries containing selected fields.
- `values_list()` returns tuples by default, or a single-value iterable with `flat=True` .

10.4 `exists()` , `count()` , and `distinct()`

```
Student.objects.filter(is_active=True).exists()

Student.objects.count()

Post.objects.values("author_id").distinct()
```

- `exists()` : checks whether any matching records exist.
- `count()` : counts matching records.
- `distinct()` : removes duplicate result rows.

Some `distinct()` behavior depends on the database backend and query structure.

Chapter 11: `select_related()` and `prefetch_related()`

11.1 The N+1 Query Problem

```
posts = Post.objects.all()

for post in posts:
    print(post.author.name)
```

Django may execute one query for posts and additional queries for individual authors.

This is an example of the N+1 query problem.

11.2 `select_related()`

Use it mainly for `ForeignKey` and `OneToOneField` relationships.

```
posts = Post.objects.select_related("author")

for post in posts:
    print(post.author.name)
```

It uses SQL JOINs to retrieve related single-valued objects efficiently.

11.3 `prefetch_related()`

Use it mainly for related collections.

```
authors = Author.objects.prefetch_related("posts")

for author in authors:
    for post in author.posts.all():
        print(post.title)
```

It normally uses additional queries and matches the results in Python.

You can combine them:

```
posts = (
    Post.objects
    .select_related("author")
    .prefetch_related("tags")
)
```

This assumes `Post` has a `tags` relationship.

Method	Typical use	Approach
`select_related()`	ForeignKey, OneToOne	SQL JOIN
`prefetch_related()`	Reverse ForeignKey, ManyToMany, collections	Additional query/queries and Python-side matching

বাংলা: N+1 query কমাতে সম্পর্ক অনুযায়ী `select_related()` বা `prefetch_related()` ব্যবহার করো। Query count মেপে ফলাফল যাচাই করবে।

Chapter 12: Aggregation and Annotation

12.1 `aggregate()`

`aggregate()` returns summary values for the whole QuerySet.

```
from django.db.models import Avg, Max, Min

result = Student.objects.aggregate(
    average_age=Avg("age"),
    oldest=Max("age"),
    youngest=Min("age"),
)
```

It returns a dictionary. The values depend on the actual data.

12.2 `annotate()`

`annotate()` adds calculated values to each returned object.

```
from django.db.models import Count

authors = Author.objects.annotate(
    post_count=Count("posts")
)

for author in authors:
    print(author.name, author.post_count)
```

12.3 Filter by Annotation

```
authors = (
    Author.objects
    .annotate(post_count=Count("posts"))
    .filter(post_count__gte=5)
)
```

Some joins can duplicate rows. Depending on the query, `Count("posts", distinct=True)` may be needed.

বাংলা:

- `aggregate()` : পুরো QuerySet-এর summary দেয়।
- `annotate()` : প্রতিটি object-এর সঙ্গে calculated value যোগ করে।

Chapter 13: F Expressions

13.1 What Is `F()`?

An `F()` expression refers to a database field inside a query. It allows database-side calculations and field comparisons.

```
from django.db.models import F
```

Example 1: Decrease Stock

```
Product.objects.filter(
    stock__gt=0
).update(
    stock=F("stock") - 1
)
```

Conceptually, the database executes an update like:

```
UPDATE products_product
SET stock = stock - 1
WHERE stock > 0;
```

The calculation happens in the database instead of first reading the old value into Python. This can avoid common read-modify-write race conditions. More complex inventory rules may still need transactions, constraints, or locking.

Example 2: Compare Fields

```
Employee.objects.filter(
    salary__gt=F("manager__salary")
)
```

Assuming `Employee` has a manager relationship, this compares an employee's salary with their manager's salary.

Example 3: Use F on an Instance

```
product = Product.objects.get(pk=1)

product.stock = F("stock") - 1

product.save(update_fields=["stock"])

product.refresh_from_db()
```

Refresh the instance before treating the field as a normal Python value.

Interview answer: `F()` refers to a database field in a query and enables database-side calculations and field comparisons.

বাংলা: `F()` ব্যবহার করলে পুরোনো value Python-এ এনে calculation করার বদলে database-কে সরাসরি calculation করতে বলা যায়।

Chapter 14: Conditional Expressions

14.1 `Case()` and `When()`

- `When()`: defines a condition.
- `then`: value when the condition matches.
- `default`: fallback value.
- `output_field`: output data type.

```
from django.db.models import Case, When, Value, CharField

students = Student.objects.annotate(
    age_group=Case(
        When(age__lt=18, then=Value("minor")),
        default=Value("adult"),
        output_field=CharField(),
    )
)
```

Each returned `Student` gets an `age_group` calculated according to the condition.

14.2 Database Functions

```
from django.db.models.functions import Lower, Upper, Length

students = Student.objects.annotate(
    lower_name=Lower("name"),
    upper_name=Upper("name"),
    name_length=Length("name"),
)
```

These functions calculate lowercase text, uppercase text, and string length in the database.

14.3 `Coalesce()`

`Coalesce()` returns the first non-NULL expression.

```
from django.db.models import Value, CharField
from django.db.models.functions import Coalesce

students = Student.objects.annotate(
    display_name=Coalesce(
        "nickname",
        "name",
        Value("Unknown", output_field=CharField()),
    )
)
```

This assumes `Student` has a nullable `nickname` field. Expressions should have compatible data types.

বাংলা: Conditional expressions দিয়ে SQL query-এর মধ্যেই শর্ত অনুযায়ী value তৈরি করা যায়।

Chapter 15: Subquery and Exists

15.1 `Exists()`

Use `Exists()` to check whether related records exist.

```
from django.db.models import Exists, OuterRef

posts = Post.objects.filter(
    author=OuterRef("pk")
)

authors = Author.objects.annotate(
    has_posts=Exists(posts)
)
```

`OuterRef("pk")` refers to the primary key of the outer `Author` query. `has_posts` indicates whether a matching `Post` exists.

To return only authors with posts:

```
authors = Author.objects.filter(Exists(posts))
```

15.2 `Subquery()`

Get the latest post title for each author:

```
from django.db.models import OuterRef, Subquery

latest_post = (
    Post.objects
    .filter(author=OuterRef("pk"))
    .order_by("-created_at", "-pk")
)

authors = Author.objects.annotate(
    latest_post_title=Subquery(
        latest_post.values("title")[:1]
    )
)
```

This assumes `Post` has a `created_at` field.

Explanation:

1. `OuterRef("pk")` identifies the current author.
2. `filter()` selects that author's posts.

3. `order_by()` puts the latest post first.
4. `values("title")` selects one column.
5. `[:1]` limits the subquery to one row.

The subquery must return the number of rows and columns required by the expression.

বাংলা: `Exists()` সম্পর্কিত record আছে কি না পরীক্ষা করে। `Subquery()` অন্য query-এর result-কে আরেকটি query-তে ব্যবহার করতে দেয়।

Chapter 16: Managers and Custom QuerySets

16.1 What Is a Manager?

A Manager provides an interface for database queries on a model. `objects` is the common default Manager.

```
Student.objects.all()
```

16.2 Custom QuerySet

```
from django.db import models


class PublishedPostQuerySet(models.QuerySet):
    def published(self):
        return self.filter(is_published=True)

    def by_author(self, author):
        return self.filter(author=author)


class Post(models.Model):
    title = models.CharField(max_length=200)
    is_published = models.BooleanField(default=False)

    objects = PublishedPostQuerySet.as_manager()
```

Usage:

```
Post.objects.published()

Post.objects.published().by_author(author)
```

Be careful when adding default filters to a Manager. They can hide records from admin pages or other workflows.

বাংলা: Reusable query logic custom QuerySet-এ রাখলে code পরিষ্কার হয় এবং একই query বারবার লেখার প্রয়োজন কমে।

Chapter 17: Database Transactions

17.1 `transaction.atomic()`

A transaction groups database operations into one logical unit.

```
from django.db import transaction

with transaction.atomic():
    order = Order.objects.create(
        customer=customer,
        total=1000,
    )

    OrderItem.objects.create(
        order=order,
        product=product,
        quantity=2,
    )
```

This assumes the `Order` and `OrderItem` models have the required fields.
If an uncaught exception causes the transaction to roll back, the database changes in that transaction can be undone. External actions such as sending an email cannot automatically be rolled back.

17.2 Why Use Transactions?

Suppose an order is created but inventory is not updated. A transaction helps avoid unwanted partial database changes.
Concurrency control and constraints may also be necessary.

17.3 `transaction.on_commit()`

```
from django.db import transaction

transaction.on_commit(
    lambda: print("Transaction committed")
)
```

The callback runs after a successful commit. If there is no active transaction, it normally executes immediately.

For important background work, a durable task queue or transactional outbox pattern may be needed because a process can stop after commit but before the callback's work completes.

বাংলা: একাধিক সম্পর্কিত database operation একসঙ্গে সফল বা ব্যর্থ হওয়া দরকার হলে transaction ব্যবহার করো।

Chapter 18: Raw SQL

Use the ORM for most queries. Raw SQL can help with special or database-specific queries.

18.1 `raw()`

```
students = Student.objects.raw(
    """
    SELECT *
    FROM students_student
    WHERE age >= %s
    """,
    [18],
)
```

The table name must match the actual database schema. A raw model query normally needs the model's primary key in the selected columns.

18.2 Cursor

```
from django.db import connection

with connection.cursor() as cursor:
    cursor.execute(
        """
        SELECT COUNT(*)
        FROM students_student
        WHERE age >= %s
        """,
        [18],
    )

    row = cursor.fetchone()
    print(row[0])
```

Always use parameterized queries. Never join untrusted user input directly into an SQL string.

Unsafe pattern — do not use:

```
# Do not do this.
# query = f"SELECT * FROM students WHERE name = '{user_input}'"
```

বাংলা: Raw SQL ব্যবহার করলেও parameterized query ব্যবহার করতে হবে, নইলে SQL injection-এর ঝুঁকি তৈরি হতে পারে। Dynamic field names-এর জন্য allowlist দরকার।

Chapter 19: Query Optimization and Indexes

19.1 Query Plan

```
students = Student.objects.filter(age__gte=18)

print(students.explain())
```

`explain()` helps inspect how the database plans to execute a query. It may show scans, index use, joins, and other operations.

The plan depends on the database, data, and configuration.

19.2 When to Use Indexes

Consider indexes for:

- Frequently filtered fields.
- Join keys.
- Frequently ordered fields.
- Common combinations of fields used by queries.

Do not add indexes to every field. Indexes use storage and can slow down writes.

19.3 Composite Index

```
class Student(models.Model):
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        indexes = [
            models.Index(
                fields=["is_active", "created_at"],
                name="student_active_created_idx",
```

```
            ),
        ]
```

The order of fields in a composite index matters. A B-tree composite index is often most useful for queries matching its leading fields, but real query plans should guide decisions.

19.4 Optimization Checklist

1. Reduce N+1 queries.
2. Use `select_related()` and `prefetch_related()` where appropriate.
3. Load only the fields you need.
4. Paginate large result sets.
5. Measure query counts and response times.
6. Inspect query plans.
7. Add indexes based on real query patterns.

বাংলা: অনুমান করে optimization না করে query count, response time ও execution plan মেপে সিদ্ধান্ত নাও।

Chapter 20: Bulk Operations

Bulk operations process many records efficiently.

20.1 `bulk_create()`

```
students = [
    Student(name="A", email="a@example.com"),
    Student(name="B", email="b@example.com"),
]

Student.objects.bulk_create(
    students,
    batch_size=500,
)
```

20.2 `bulk_update()`

```
students = list(
    Student.objects.filter(is_active=False)
)

for student in students:
```

```
    student.is_active = True

Student.objects.bulk_update(
    students,
    ["is_active"],
    batch_size=500,
)
```

Bulk operations generally do not call each instance's `save()` method or normal per-instance save signals. Backend behavior can vary.

Feature	Normal `save()`	Bulk operations
Instance save behavior	Runs	Usually bypassed
Normal save signals	Applicable	Generally not sent per instance
Many records	Can have more overhead	Often more efficient
Per-object custom logic	Straightforward	May need separate handling

বাংলা: অনেক record একসঙ্গে insert/update করতে bulk method কার্যকর, তবে validation ও business logic কোথায় হবে তা নিশ্চিত করতে হবে।

Chapter 21: Multiple Databases

Django can connect to multiple databases.

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    },
    "analytics": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "analytics.sqlite3",
    },
}
```

Read from a specific database:

```
Student.objects.using("analytics").all()
```

Save to a specific database:

```
student.save(using="analytics")
```

Multiple databases may require database routers and careful migration planning. Cross-database relationships and transactions have limitations; one atomic transaction does not automatically cover every database.

বাংলা: একাধিক database থাকলে কোন model কোথায় read/write হবে, migration কীভাবে চলবে এবং transaction-এর সীমাবদ্ধতা কী—এসব পরিকল্পনা করতে হয়।

Chapter 22: Fixtures

Fixtures are files containing data that can be loaded into a database.

Export Data

```
python manage.py dumpdata students --indent 2 > students.json
```

Import Data

```
python manage.py loaddata students.json
```

Fixtures can help create sample data, demo records, or known test data.

Consider primary keys, model dependencies, and environment-specific data. Remove private information before sharing exports.

বাংলা: Fixture development ও testing-এ নির্দিষ্ট data পুনরায় load করার কাজে লাগে। Sensitive production data অসতর্কভাবে export/share করবে না।

Chapter 23: Django Signals

23.1 What Are Django Signals?

Django Signals allow certain senders to notify registered receivers when specific events occur.

In simple terms, a receiver function can execute automatically when a particular event happens.

Main components:

- **Sender:** The model or process that triggers an event.
- **Signal:** The notification sent when an event occurs.
- **Receiver:** The function that responds to the signal.
- **Dispatcher:** The mechanism that connects senders and receivers.

Basic workflow:

```
Event Occurs
     |
     v
Signal Is Sent
     |
     v
Registered Receiver Executes
```

Signals use a concept similar to callback functions.

23.2 Python Callback Example

```
def send_notification(username):
    print(f"Welcome, {username}!")


def create_user(username, callback):
    print(f"User {username} created.")
    callback(username)


create_user("Faruk", send_notification)
```

Output:

```
User Faruk created.
Welcome, Faruk!
```

Here, `send_notification` is a callback function passed to `create_user`.

23.3 Important Built-in Django Signals

Signal	Purpose
`pre_save`	Sent before a model instance is saved
`post_save`	Sent after a model instance is saved
`pre_delete`	Sent before a model instance is deleted
`post_delete`	Sent after a model instance is deleted
`m2m_changed`	Sent when a many-to-many relationship changes
`pre_init`	Sent before a

	model instance is initialized
`post_init`	Sent after a model instance is initialized
`request_started`	Sent when Django begins handling a request
`request_finished`	Sent when Django finishes handling a request

23.4 `pre_save` vs `post_save`

`pre_save`

The `pre_save` signal is sent before Django saves a model instance.

```
from django.db.models.signals import pre_save
from django.dispatch import receiver
from django.contrib.auth.models import User


@receiver(pre_save, sender=User)
def before_user_save(sender, instance, **kwargs):
    print("User is about to be saved.")
```

`post_save`

The `post_save` signal is sent after Django saves a model instance.

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.contrib.auth.models import User


@receiver(post_save, sender=User)
def after_user_save(sender, instance, created, **kwargs):
    if created:
        print("A new user was created.")
    else:
        print("An existing user was updated.")
```

Important: `post_save` does not guarantee that the surrounding database transaction has committed.

23.5 Signal Project Setup

Assume the project is named `config` and the application is named `accounts`.

Project structure:

```
myproject/
├── config/
│   ├── settings.py
│   └── urls.py
├── accounts/
│   ├── apps.py
│   ├── models.py
│   ├── signals.py
│   ├── views.py
│   └── tests.py
└── manage.py
```

Create the app:

```
python manage.py startapp accounts
```

Register the app in `settings.py`:

```
INSTALLED_APPS = [
    # Other apps...
```

```
    "accounts.apps.AccountsConfig",
]
```

Create `accounts/signals.py`:

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.contrib.auth.models import User


@receiver(post_save, sender=User)
def user_saved(sender, instance, created, **kwargs):
    if created:
        print(f"New user created: {instance.username}")
```

Register signals in `accounts/apps.py`:

```
from django.apps import AppConfig


class AccountsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "accounts"

    def ready(self):
        import accounts.signals
```

Django calls `ready()` during application initialization. Importing the signals module here ensures that its receiver functions are registered.

Creating `signals.py` alone is not enough; the module must be imported.

Best practices:

- Avoid database queries inside `ready()`.
- Use `dispatch_uid` when needed to prevent duplicate receiver registration.
- Keep receivers small and focused.

23.6 Automatically Create a User Profile

This is a common real-world use of Django Signals.

Step 1: Create the Profile Model

In `accounts/models.py`:

```
from django.db import models
from django.contrib.auth.models import User
```

```
class Profile(models.Model):
    user = models.OneToOneField(
        User,
        on_delete=models.CASCADE,
        related_name="profile",
    )
    bio = models.TextField(blank=True)
    phone = models.CharField(max_length=15, blank=True)

    def __str__(self):
        return self.user.username
```

The `OneToOneField` ensures that each user can have at most one associated profile.

Step 2: Create the Signal

In `accounts/signals.py`:

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.contrib.auth.models import User
from .models import Profile


@receiver(post_save, sender=User)
def create_user_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)
```

Step 3: Run Migrations

```
python manage.py makemigrations
python manage.py migrate
```

Step 4: Test in the Django Shell

```
python manage.py shell
```

```
from django.contrib.auth.models import User
from accounts.models import Profile

user = User.objects.create_user(
    username="faruk",
    password="strong-test-password",
)
```

```
profile = Profile.objects.get(user=user)

print(profile.user.username)
```

Output:

```
faruk
```

Why check `created`?

- `created=True`: a new database record was created.
- `created=False`: an existing record was saved.

Checking `created` prevents the receiver from attempting to create another profile every time an existing user is updated.

Alternative:

```
@receiver(post_save, sender=User)
def ensure_user_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.get_or_create(user=instance)
```

This helps avoid creating a duplicate profile if one already exists. The database uniqueness constraint from `OneToOneField` remains important.

23.7 `pre_save` Example: Normalize a Product Name

In `shop/models.py`:

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=200)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )
```

In `shop/signals.py`:

```
from django.db.models.signals import pre_save
from django.dispatch import receiver
from .models import Product
```

```
@receiver(pre_save, sender=Product)
def normalize_product_name(sender, instance, **kwargs):
    instance.name = instance.name.strip()
```

Usage:

```
product = Product(
    name=" Wireless Mouse ",
    price=500,
)

product.save()

print(product.name)
```

Output:

```
Wireless Mouse
```

The receiver removes leading and trailing whitespace before the product is saved.

For simple model-specific normalization, a `save()` override or explicit validation may be clearer than a signal.

23.8 `pre_delete` and `post_delete`

- `pre_delete`: sent before the delete operation.
- `post_delete`: sent after the delete operation.

```
from django.db.models.signals import pre_delete, post_delete
from django.dispatch import receiver
from .models import Product


@receiver(pre_delete, sender=Product)
def before_product_delete(sender, instance, **kwargs):
    print(f"Deleting product: {instance.name}")


@receiver(post_delete, sender=Product)
def after_product_delete(sender, instance, **kwargs):
    print(f"Deleted product: {instance.name}")
```

After deletion, the database row no longer exists, but the in-memory instance can still provide its field values.

23.9 `m2m_changed` Signal

The `m2m_changed` signal is sent when a many-to-many relationship changes through Django's relationship manager.

Example models:

```
from django.db import models


class Category(models.Model):
    name = models.CharField(max_length=100)


class Product(models.Model):
    name = models.CharField(max_length=200)
    categories = models.ManyToManyField(
        Category,
        related_name="products",
        blank=True,
    )
```

Signal receiver:

```
from django.db.models.signals import m2m_changed
from django.dispatch import receiver
from .models import Product


@receiver(
    m2m_changed,
    sender=Product.categories.through,
)
def product_categories_changed(
    sender,
    instance,
    action,
    reverse,
    model,
    pk_set,
    **kwargs,
):
    if action == "post_add":
        print("Categories added.")
    elif action == "post_remove":
```

```
            print("Categories removed.")
        elif action == "post_clear":
            print("All categories cleared.")
```

Important actions:

Action	Meaning
`pre_add`	Before relationships are added
`post_add`	After relationships are added
`pre_remove`	Before relationships are removed
`post_remove`	After relationships are removed
`pre_clear`	Before all relationships are cleared
`post_clear`	After all relationships are cleared

The value of `pk_set` depends on the action and relationship direction. It may be `None` for clear operations.

23.10 Custom Signals

Custom signals allow you to define application-specific events.

In `orders/events.py`:

```
import django.dispatch

order_placed = django.dispatch.Signal()
```

In `orders/receivers.py`:

```
from django.dispatch import receiver
from .events import order_placed


@receiver(order_placed)
def handle_order_placed(sender, order, **kwargs):
    print(f"Order placed: {order.id}")
```

Import the receiver module during application initialization to register the receiver.

In `orders/services.py`:

```
from .events import order_placed


def notify_order_placed(order):
    order_placed.send(
        sender=order.__class__,
        order=order,
    )
```

Usage:

```
order = ...  # An existing Order instance

notify_order_placed(order)
```

The receiver receives `order` as a keyword argument.

Custom signals are useful for:

- Notifying independent components about an event.
- Providing extension points in reusable applications.

- Connecting multiple independent receivers to one event.

For workflows requiring strict execution order, explicit service functions are often easier to maintain.

23.11 Django Signals vs Model `save()` Override

Feature	Signals	`save()` Override
Purpose	Event handling	Customize model save behavior
Multiple independent handlers	Supported	Not directly
Code location	Can be in a separate module	Usually in the model class
Execution flow	Requires locating the receiver	Visible in the model class
Typical use	Independent side effects	Model-specific save rules

Example:

```
class Product(models.Model):
    name = models.CharField(max_length=200)
```

```
    def save(self, *args, **kwargs):
        self.name = self.name.strip()
        super().save(*args, **kwargs)
```

For a model's own normalization rules, a `save()` override or validation logic may be clearer than a signal.

23.12 `transaction.on_commit()` with Signals

Use `transaction.on_commit()` to execute a callback after the active database transaction successfully commits.

Example: Send an email after an order is created.

```
from django.db import transaction
from django.db.models.signals import post_save
from django.dispatch import receiver
from .models import Order


def send_order_email(order_id):
    print(f"Sending email for order {order_id}")


@receiver(post_save, sender=Order)
def order_created(sender, instance, created, **kwargs):
    if created:
        transaction.on_commit(
            lambda: send_order_email(instance.pk)
        )
```

If the transaction rolls back, the callback is discarded. If there is no active transaction, the callback normally executes immediately.

Common uses:

- Sending emails.
- Queuing background tasks.
- Invalidating caches.
- Notifying external services.

For critical business events, also consider failure handling, retries, and the transactional outbox pattern.

23.13 Django Signals with Celery

Heavy tasks can be delegated to a background worker using Celery.

In `products/tasks.py`:

```
from celery import shared_task


@shared_task
def process_product_image(product_id):
    print(f"Processing product image: {product_id}")
```

In `products/signals.py`:

```
from django.db import transaction
from django.db.models.signals import post_save
from django.dispatch import receiver
from .models import Product
from .tasks import process_product_image


@receiver(post_save, sender=Product)
def product_saved(sender, instance, created, **kwargs):
    if created:
        product_id = instance.pk

        transaction.on_commit(
            lambda: process_product_image.delay(product_id)
        )
```

Workflow:

1. Django Signal detects the event.
2. `transaction.on_commit()` schedules the callback after a successful commit.
3. Celery executes the task in a background worker.

This pattern does not guarantee exactly-once task execution. Production systems should consider retries, idempotency, and task-delivery failures.

23.14 Django REST Framework and Signals

Django model signals also work when a model instance is created through Django REST Framework, provided the operation uses the normal model save lifecycle.

```
from rest_framework import serializers
from django.contrib.auth.models import User


```

```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ["id", "username", "password"]
        extra_kwargs = {
            "password": {"write_only": True}
        }

    def create(self, validated_data):
        return User.objects.create_user(**validated_data)
```

When `create_user()` creates a `User`, the registered `post_save` receiver also runs. However, operations such as `bulk_create()` and `QuerySet.update()` generally bypass the normal per-instance save signals.

23.15 Important Signal Parameters

Parameter	Meaning
`sender`	The model class that sent the signal
`instance`	The affected model instance
`created`	Whether a new record was created
`raw`	Whether the save

	is occurring in raw mode
`using`	The database alias used
`update_fields`	The set of fields being updated, or `None`

Not every signal provides the same parameters. Write receiver signatures according to the signal being used.

23.16 Common Mistakes and Best Practices

Mistake 1: Not Importing the Signals Module

Creating `signals.py` does not automatically register its receivers.

Solution: Import the signals module from `AppConfig.ready()`.

Mistake 2: Infinite Recursion

Avoid this pattern:

```
@receiver(post_save, sender=Product)
def bad_signal(sender, instance, **kwargs):
    instance.save()
```

Calling `instance.save()` inside the same model's `post_save` receiver can trigger the receiver again.

Mistake 3: Putting Too Much Business Logic in Signals

Avoid placing long workflows directly inside signal receivers.

Better approaches include service functions or Celery tasks where appropriate.

Important Limitations

- `bulk_create()` generally does not trigger `pre_save` or `post_save`.

- `QuerySet.update()` does not call each model's `save()` method or its save signals.
- `post_save` does not guarantee transaction commit.
- Receiver exceptions can cause the calling operation to fail.
- During raw fixture loading, avoid modifying other database objects unless the operation is specifically designed for it.

Testing Signals

In `accounts/tests.py`:

```
from django.test import TestCase
from django.contrib.auth.models import User
from .models import Profile


class UserProfileSignalTest(TestCase):
    def test_profile_created_with_user(self):
        user = User.objects.create_user(
            username="faruk",
            password="test-password",
        )

        self.assertTrue(
            Profile.objects.filter(user=user).exists()
        )
```

Run the tests:

```
python manage.py test accounts
```

Additional tests:

- Verify that updating a user does not create a duplicate profile.
- Verify that the receiver is registered.
- Verify that transaction rollback discards an `on_commit()` callback.
- Test expected behavior when a receiver raises an exception.

Django Signals Interview Questions

Q1. What are Django Signals?

They allow senders to notify registered receivers when specific events occur.

Q2. Difference between `pre_save` and `post_save`?

`pre_save` is sent before the save operation; `post_save` is sent after it.

Q3. What does `created=True` mean?

A new database record was created.

Q4. Why use `AppConfig.ready()`?

It provides a standard place to import the signals module and register receivers during application initialization.

Q5. Can signals cause recursion?

Yes. Calling `save()` inside the same model's `post_save` receiver can trigger the receiver repeatedly.

Q6. Do `bulk_create()` and `QuerySet.update()` trigger save signals?

Generally, no.

Q7. What is `transaction.on_commit()`?

It registers a callback that executes after the active database transaction successfully commits.

Q8. Can multiple receivers listen to the same signal?

Yes.

Q9. What is a custom signal?

A signal defined by an application for its own events using `django.dispatch.Signal`.

Q10. When should you avoid signals?

Avoid them when an explicit service call is clearer or when execution order, error handling, and return values need explicit control.

Chapter 24: Abstract Models and Proxy Models

24.1 Abstract Base Model

An abstract model is used for model inheritance and code reuse.

It:

- Does not create its own database table.
- Allows multiple models to reuse common fields and methods.
- Passes inherited fields and methods to child models.

Use it when multiple models share common fields or behavior.

Example

```
from django.db import models
```

```
class CommonInfo(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True


class Student(CommonInfo):
    name = models.CharField(max_length=100)
    age = models.IntegerField()


class Teacher(CommonInfo):
    name = models.CharField(max_length=100)
    subject = models.CharField(max_length=100)


class Employee(CommonInfo):
    name = models.CharField(max_length=100)
    salary = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )
```

Explanation

- `CommonInfo` does not create a database table.
- Each concrete child model gets its own database table.
- The child tables include the inherited fields.
- `abstract = True` makes `CommonInfo` an abstract base model.

Abstract models are useful for reusing common fields and methods across multiple models.

24.2 Proxy Model

A proxy model reuses the original concrete model's database table. It does not introduce new database fields.

It can add or override:

- Python methods.
- Default ordering.
- Managers.

- Python-level behavior.

Example

```
from django.db import models


class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()

    class Meta:
        ordering = ["name"]


class StudentProxy(Student):
    class Meta:
        proxy = True
        ordering = ["age"]

    def is_adult(self):
        return self.age >= 18
```

The `proxy = True` option defines a proxy model.

Usage:

```
student = StudentProxy.objects.get(id=1)

print(student.name)
print(student.is_adult())
```

`StudentProxy` accesses the same student records as `Student`. It can have different default ordering and additional Python methods without adding database fields.

Use proxy models when you want to customize model behavior without changing the database structure.

24.3 Abstract Model vs Proxy Model

Feature	Abstract Model	Proxy Model
Database table for	No table for the	Reuses the

the parent	abstract parent	concrete model's table
Field inheritance	Fields are inherited into child models	Cannot add new database fields
Methods	Can define reusable methods	Can add or override methods
Purpose	Reuse common fields and behavior	Customize existing model behavior
Database structure	Each concrete child gets its own table	No additional model table
Meta option	`abstract = True`	`proxy = True`
Typical use case	Shared timestamps or common fields	Custom ordering, methods, or managers

Remember:

```
Abstract Model = Reuse fields and methods.
Proxy Model    = Reuse the existing table and customize behavior.
```

Chapter 25: Validation, Constraints and Security

25.1 Model Validation

```
student = Student(
    name="",
    email="invalid-email",
)

student.full_clean()
```

`full_clean()` performs field, model, and constraint-related validation where applicable, and may raise `ValidationError`.
Django's `save()` does not automatically call `full_clean()`. Decide where validation should occur. In Django REST Framework applications, serializer validation is also important.

25.2 CheckConstraint

Example: Product price must not be negative.

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    class Meta:
        constraints = [
            models.CheckConstraint(
                condition=models.Q(price__gte=0),
                name="product_price_nonnegative",
            )
        ]
```

This creates a database-level rule that the price must be at least zero.

25.3 UniqueConstraint

Prevent duplicate enrollment:

```
class Enrollment(models.Model):
    learner = models.ForeignKey(
        "Learner",
        on_delete=models.CASCADE,
    )
    course = models.ForeignKey(
        "Course",
        on_delete=models.CASCADE,
    )

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["learner", "course"],
                name="unique_enrollment",
            )
        ]
```

A database constraint protects data integrity even when concurrent requests try to create duplicates.

বাংলা: শুধু form বা serializer validation যথেষ্ট নয়। Data integrity-এর জন্য প্রয়োজন অনুযায়ী database constraint-ও ব্যবহার করো।

25.4 Security Checklist

- Use parameterized queries for raw SQL.
- Check user permissions and object-level authorization.
- Keep credentials out of source code.
- Set `DEBUG=False` in production.
- Configure `ALLOWED_HOSTS` correctly.
- Do not log sensitive information.
- Give database users only the permissions they need.
- Test backup and restore procedures.
- Allowlist user-selectable ordering fields and column names.
- Test authentication and authorization on protected API endpoints.

Chapter 26: Python and Django Introspection

26.1 What Is Introspection?

Introspection means examining an object or class at runtime to find information about it.

In Python and Django, introspection can help you find:

- What type an object is.
- What class it belongs to.
- Which module defines the class.
- Django model name.
- Django app name.
- Database table name.
- Model fields.
- Model relationships.

Important Python tools:

```
type()
isinstance()
__class__
```

For Django models:

```
_meta
```

26.2 Sample Django Models

Suppose the app is named `shop`.

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.IntegerField()


class Customer(models.Model):
    email = models.EmailField()
```

Get objects from the database:

```
product = Product.objects.get(id=1)
customer = Customer.objects.get(id=1)
```

26.3 What Is `type()`?

`type()` is a built-in Python function. It tells you the type/class of an object.

```
print(type(product))
```

Possible output:

```
<class 'shop.models.Product'>
```

The exact output depends on your project's module path.

Get the Class Name

```
print(type(product).__name__)
```

Output:

```
Product
```

Get the Module

```
print(type(product).__module__)
```

Output:

```
shop.models
```

26.4 What Is `__class__`?

Python objects also have a `__class__` attribute.

```
print(product.__class__)
print(product.__class__.__name__)
print(product.__class__.__module__)
```

In ordinary Python code, `type(product)` and `product.__class__` both refer to the object's class.

For readability, `type()` is usually the more familiar choice when you simply want to inspect an object's type.

26.5 What Is Django `_meta`?

Django models have a metadata API called `_meta`.

```
product._meta
```

It provides information such as:

- Model name.

- Class name.
- App label.
- Database table.
- Fields.
- Primary key.
- Relationships.
- Model options.

This is especially useful for generic and reusable Django code.

`_meta.model_name`

```
product._meta.model_name
```

Output:

```
product
```

`model_name` is generally the lowercase model name.

`_meta.object_name`

```
product._meta.object_name
```

Output:

```
Product
```

Difference:

```
_meta.model_name   -> product
_meta.object_name  -> Product
```

`_meta.app_label`

```
product._meta.app_label
```

Possible output:

```
shop
```

`_meta.db_table`

```
product._meta.db_table
```

Possible output:

```
shop_product
```

The exact table name depends on your model configuration and Django app/model names.

26.6 `_meta.get_fields()`

You can inspect a model's fields and relationships:

```
for field in Product._meta.get_fields():
    print(field.name)
```

Possible output:

```
id
name
price
```

The exact list depends on the model. `get_fields()` can include relationship-related fields as well as ordinary concrete fields.

26.7 Real-Life Example: Generic Model Information

```
def show_info(obj):
    print("App:", obj._meta.app_label)
    print("Model:", obj._meta.model_name)
    print("Class Name:", obj._meta.object_name)
```

Usage:

```
product = Product.objects.get(id=1)
customer = Customer.objects.get(id=1)

show_info(product)
show_info(customer)
```

Possible output:

```
App: shop
Model: product
Class Name: Product

App: shop
Model: customer
Class Name: Customer
```

The function is not specifically written for `Product`. It can work with many different Django model instances.

26.8 Dynamic JSON API Example

```
from django.http import JsonResponse


def get_data(request):
    product = Product.objects.get(id=1)

    data = {
        "model": product._meta.model_name,
        "app": product._meta.app_label,
        "name": product.name,
    }

    return JsonResponse(data)
```

Response:

```
{
    "model": "product",
    "app": "shop",
    "name": "Laptop"
}
```

Here, the model name and app label are obtained dynamically from the Django model.

26.9 `isinstance()`

`isinstance()` checks whether an object is an instance of a particular class or one of its subclasses.

Syntax:

```
isinstance(object, Class)
```

Example:

```
product = Product.objects.get(id=1)

print(isinstance(product, Product))
```

Result:

```
True
```

For another model:

```
customer = Customer.objects.get(id=1)

print(isinstance(customer, Product))
```

Result:

```
False
```

26.10 `type()` vs `isinstance()`

Consider:

```
class Animal:
    pass


class Dog(Animal):
    pass


dog = Dog()
```

Using `type()`:

```
type(dog) is Dog
# True

type(dog) is Animal
# False
```

Using `isinstance()`:

```
isinstance(dog, Dog)
# True

isinstance(dog, Animal)
# True
```

`Dog` inherits from `Animal`, so a `Dog` instance is also an instance of `Animal`.

Remember:

```
type()        -> Check the exact type.
isinstance()  -> Check class/subclass relationship.
```

26.11 Generic Logging Example

```
def log_object(obj):
    print(
        f"[{obj._meta.app_label}] "
        f"{obj._meta.object_name} loaded"
    )
```

For a product:

```
[shop] Product loaded
```

For a customer:

```
[shop] Customer loaded
```

The same function works with different Django models.

26.12 Generic Save Message

```
def save_message(obj):
    return (
        f"{obj._meta.object_name} "
        f"saved successfully!"
    )
```

Usage:

```
print(save_message(product))
print(save_message(customer))
```

Possible output:

```
Product saved successfully!
Customer saved successfully!
```

26.13 Generic Model Information Dictionary

```
def get_model_info(obj):
    return {
        "app": obj._meta.app_label,
        "model": obj._meta.model_name,
        "class": obj._meta.object_name,
        "table": obj._meta.db_table,
    }
```

For a product, a possible result is:

```
{
    "app": "shop",
    "model": "product",
    "class": "Product",
    "table": "shop_product",
}
```

26.14 Inspecting Model Fields

```
def get_fields(obj):
    return [
        field.name
        for field in obj._meta.get_fields()
    ]
```

Usage:

```
print(get_fields(product))
```

Possible output for the simple model:

```
['id', 'name', 'price']
```

This technique can be useful for:

- Generic admin tools.
- Debugging tools.
- Generic serializers.
- Reusable utilities.
- Dynamic APIs.
- Model inspection.

26.15 Python Introspection vs Django Introspection

Direct Python Introspection

```
product.__class__

product.__class__.__name__

product.__class__.__module__
```

Python Built-in Functions

```
type(product)

type(product).__name__

type(product).__module__
```

Django Model Introspection

```
product._meta.model_name

product._meta.object_name

product._meta.app_label

product._meta.db_table

product._meta.get_fields()
```

26.16 When Should You Use Each One?

Requirement	Use
Python object type	`type(obj)`
Python class name	`type(obj).__name__`
Python module	`type(obj).__module__`
Check class/subclass	`isinstance(obj, SomeClass)`
Get object's	`obj.__class__`

class	
Django model name	`obj._meta.model_name`
Django class name	`obj._meta.object_name`
Django app label	`obj._meta.app_label`
Database table	`obj._meta.db_table`
Model fields/relationships	`obj._meta.get_fields()`

It is better not to say `_meta` must be used for every dynamic system.
A more accurate statement is:

> `_meta` is a powerful and standard Django API for dynamically inspecting Django model metadata.

If you only need the Python class name, `type(obj).__name__` is enough. If you need Django-specific information such as app label, model name, database table, fields, or relationships, `_meta` is appropriate.

Quick Mental Model

```
Python Object
    |
    +-- type()
    +-- isinstance()
    +-- __class__

Django Model
    |
```

```
      +-- _meta.model_name
      +-- _meta.object_name
      +-- _meta.app_label
      +-- _meta.db_table
      +-- _meta.get_fields()
```

Most important concept:

```
type()        -> Python-level type information
isinstance()  -> Class/subclass relationship checking
_meta         -> Django model metadata/introspection
```

If you understand these three concepts separately, Django's generic functions, reusable utilities, dynamic APIs, debugging tools, and model introspection become easier to understand.

Chapter 27: Django ORM Interview Questions

Beginner Level

1. What is Django ORM?

It lets you work with a relational database through Python models and query methods.

2. What is a Model?

A Python class that defines the structure and behavior of stored data.

3. What is a Migration?

A version-controlled instruction for changing database schema.

4. Difference between `makemigrations` and `migrate`?

`makemigrations` creates migration files. `migrate` applies them to the database.

5. Difference between `get()` and `filter()`?

`get()` expects one object and may raise an exception. `filter()` returns a QuerySet.

6. What is a lazy QuerySet?

A QuerySet usually does not run its database query until its results are needed.

Intermediate Level

7. Difference between `select_related()` and `prefetch_related()`?

`select_related()` uses SQL JOINs for single-valued relationships.

`prefetch_related()` generally uses additional queries for related collections.

8. Difference between `annotate()` and `aggregate()`?

`annotate()` adds calculated values to individual objects. `aggregate()` returns a summary for the whole QuerySet.

9. What is an F expression?

It refers to a database field inside a query and allows database-side calculations or field comparisons.

10. What is a Q object?

It builds complex query conditions such as OR, AND, and NOT.

11. Difference between `null=True` and `blank=True`?

`null` affects database storage. `blank` affects validation.

12. What does CASCADE do?

It causes related objects to be deleted when the referenced object is deleted, according to the relationship's deletion behavior.

13. Should every field have an index?

No. Indexes use storage and may slow down writes. Add them based on query patterns and measurements.

Advanced Level

14. What is `transaction.atomic()`?

It groups database operations in a transaction so appropriate changes can roll back on failure.

15. Difference between `update()` and `save()`?

`update()` directly updates matching rows. `save()` saves a model instance.

16. What is the N+1 query problem?

It occurs when retrieving related data triggers additional queries for many individual objects.

17. Why use `bulk_create()`?

It can insert many records efficiently, but does not provide all normal per-instance save behavior.

18. Why use database constraints?

They protect data integrity at the database level, including against many concurrent-request problems.

19. When should you use `exists()`?

When you only need to know whether any matching record exists.

20. How do you optimize ORM queries?

Measure query counts and response times, inspect query plans, reduce N+1 queries, load only needed data, paginate results, and add indexes carefully.

Chapter 28: Quick ORM Cheat Sheet

Create

```
Student.objects.create(
    name="Rahim",
    email="rahim@example.com",
)
```

Read

```
Student.objects.all()

Student.objects.get(pk=1)

Student.objects.filter(age__gte=18)
```

Update

```
Student.objects.filter(pk=1).update(
    is_active=False
)
```

Delete

```
Student.objects.filter(pk=1).delete()
```

Ordering

```
Student.objects.order_by("-created_at")
```

Limit

```
Student.objects.all()[:10]
```

Exists

```
Student.objects.filter(
    is_active=True
```

```
).exists()
```

Count

```
Student.objects.count()
```

OR Condition

```
from django.db.models import Q

Student.objects.filter(
    Q(age__lt=18) | Q(age__gt=60)
)
```

F Expression

```
from django.db.models import F

Product.objects.filter(
    stock__gt=0
).update(
    stock=F("stock") - 1
)
```

Aggregation

```
from django.db.models import Count

Author.objects.aggregate(
    total=Count("id")
)
```

Annotation

```
Author.objects.annotate(
    post_count=Count("posts")
)
```

Select Related

```
Post.objects.select_related("author")
```

Prefetch Related

```
Author.objects.prefetch_related("posts")
```

Transaction

```
from django.db import transaction

with transaction.atomic():
    pass
```

Migration Commands

```
python manage.py makemigrations
python manage.py migrate
python manage.py showmigrations
```

বাংলা: এই cheat sheet coding-এর সময় দ্রুত reference হিসেবে ব্যবহার করতে পারো। তবে প্রতিটি method-এর behavior ও সীমাবদ্ধতা বুঝে ব্যবহার করবে।

Chapter 29: Learning Roadmap

Beginner Level

1. Learn tables, rows, columns, and primary keys.
2. Learn foreign keys.
3. Create Django models and fields.
4. Practice migrations.
5. Practice CRUD operations.
6. Write queries in the Django shell.
7. Learn `type()`, `isinstance()`, and basic model introspection.

Intermediate Level

1. Learn field lookups and Q objects.
2. Practice ForeignKey, OneToOne, and ManyToMany relationships.
3. Learn F expressions.
4. Practice aggregation and annotation.
5. Use `select_related()` and `prefetch_related()`.
6. Learn validation and database constraints.
7. Practice transactions.
8. Learn Django Signals and signal registration.
9. Understand abstract models and proxy models.
10. Inspect Django model metadata through `_meta`.

Advanced Level

1. Understand query plans and indexes.
2. Learn Subquery and Exists.
3. Practice bulk operations.
4. Create custom managers and QuerySets.
5. Learn multiple databases and routers.
6. Understand concurrency and transaction isolation.
7. Plan production migrations.
8. Monitor query performance.
9. Test signal behavior and transaction callbacks.
10. Learn reliable background-task delivery and the transactional outbox pattern.

Chapter 30: Capstone Project — Student Course Management API

Build a practical backend project that combines these concepts.

Models

- Student
- Course
- Instructor
- Enrollment

Requirements

- Student CRUD API.
- Course CRUD API.
- Student enrollment system.
- Prevent duplicate enrollments.
- Count students in each course.
- List courses for each student.
- Filtering and pagination.
- Query optimization.
- Transaction support.
- Automated tests.
- PostgreSQL configuration.

- API documentation.

Example: Count Students per Course

Assuming `Learner.courses` is a ManyToManyField with `related_name="learners"`:

```
from django.db.models import Count

courses = Course.objects.annotate(
    student_count=Count("learners", distinct=True)
)
```

Example: Load a Learner's Courses Efficiently

```
learner = Learner.objects.prefetch_related(
    "courses"
).get(pk=1)

for course in learner.courses.all():
    print(course.title)
```

Test Cases

- Duplicate email is rejected.
- Duplicate enrollment is prevented.
- Missing records are handled correctly.
- Unauthorized users cannot access protected data.
- N+1 queries are reduced.
- Failed transactions do not leave unwanted partial database changes.
- Signals do not create duplicate related records.
- Bulk operations behave as expected.
- Database constraints protect important business rules.

Official Django Documentation

- Django 6.0 Documentation: https://docs.djangoproject.com/en/6.0/
- Models: https://docs.djangoproject.com/en/6.0/topics/db/models/
- QuerySets: https://docs.djangoproject.com/en/6.0/ref/models/querysets/
- Query Expressions: https://docs.djangoproject.com/en/6.0/ref/models/expressions/

- Database Optimization: https://docs.djangoproject.com/en/6.0/topics/db/optimization/
- Transactions: https://docs.djangoproject.com/en/6.0/topics/db/transactions/
- Migrations: https://docs.djangoproject.com/en/6.0/topics/migrations/
- Signals: https://docs.djangoproject.com/en/6.0/topics/signals/

Final Advice

1. **Practice:** Write every example yourself.
2. **Debug:** Inspect generated SQL and query counts.
3. **Test:** Check edge cases, constraints, concurrent updates, and transaction failures.
4. **Optimize:** Measure query performance using realistic data.
5. **Build:** Apply the concepts in a complete project.
6. **Understand:** Know when to use ORM methods, signals, constraints, or explicit service functions.
7. **Review:** Use official documentation when behavior depends on the Django version or database backend.

Reading notes is not enough to master Django ORM.

Practice + Testing + Debugging = Strong Django ORM Skills

বাংলা: শুধু code মুখস্থ করবে না। কেন ব্যবহার করতে হয়, কখন ব্যবহার করতে হয়, database-এ কী প্রভাব পড়ে এবং কীভাবে test করতে হয়—এসব বুঝলেই Django ORM-এ দক্ষতা তৈরি হবে।